

AI agent review guidelines

Rules and a step-by-step procedure for AI agents that repeatedly screen the Došlap app for defects and propose improvements. Specific to this app, written to stay valid when files move or are renamed.

Version 2 (5 Oct 2026) — **highlighted** text is new since version 1 (25 Sep 2026).

Audience	An AI coding agent (e.g. Claude Code) with shell access to the repository, run on a schedule or on request. Humans can use it as a checklist too.
Output	A review report (section 9) with evidence-backed findings, improvement proposals and coverage gaps. Read-only by default.
Two depths	Quick screen (section 10, ~15–30 min) for frequent runs; full review (section 5, phases A–J) weekly, after large changes, or when the quick screen finds something.
Companion docs	The newest Documentation/Engine_Doslap_v*_dokumentace.pdf (engine design, formulas, evidence; source Documentation/engine/) and the most recent report in Documentation/reviews/ on branch claude/reviews .

0. The one principle: contracts, not paths

Everything in this playbook is written as **behavioural contracts** (invariants the app must keep) plus **discovery searches** that find the code responsible for each contract. File paths are given only as hints valid on **5 Oct 2026**. When a hint no longer matches:

1. find the code by its role with the discovery search in section 4 (search by function or concept, not by path);
2. run the check against the new location;
3. record the move under “Structural drift” in the report and propose the playbook update;
4. if you cannot find a responsible component at all, report it as a **coverage gap** — never skip a check silently.

1. Rules of engagement (hard rules)

1. **Read-only by default.** Do not edit code, data, configuration, tests or documentation unless the task explicitly asks for it. Describe fixes in the report (a diff sketch is fine).
2. **Never touch production or real people’s data.** No mutating calls to the deployed app, no real Garmin or Apple accounts, no real runner data. Run everything against a throwaway database that the app seeds itself (appendix A).
3. **No data to external AI services during review.** Leave NVIDIA_API_KEY, LLM_API_KEY, ASSISTANT_API_KEY, EMBED_API_KEY, VISION_API_KEY and ASR keys unset; the LLM features then fall back to rule-based output. Never paste runner data into web tools.
4. **Never commit, push, rebase, reset or discard** on the code branches. Uncommitted changes and untracked files belong to the owner: note them in the baseline, never clean them up. **Reports are the one exception: they are committed only to branch `claude/reviews` (never merge main into it, never force-push).**
5. **Evidence before assertion.** Every finding needs a location (file:line), a concrete failure scenario and evidence: a failing test, script output, a screenshot, or a precise code-path argument. Label it CONFIRMED (reproduced) or LIKELY / PLAUSIBLE (reasoned, not reproduced).
6. **Never weaken tests or thresholds to get green.** A failing test is a finding.
7. **Respect product decisions (section 2).** You may question them, but as a “Product question”, never as a bug.
8. **Verify every citation** (authors, title, journal, year, volume, pages, DOI and what the study actually found) against PubMed, the publisher or an institutional repository before proposing it; flag existing references that overstate their evidence.
9. **Both directions of health risk are defects.** False reassurance (missing a real overload, suppressing a needed physio referral) and false alarms (alarm fatigue, needless referrals) both count; say which one a finding causes.
10. **Czech is the product language.** Judge UI copy by Czech conventions (formal “vy”); the English UI is a translation layer (section 3) — check it for untranslated Czech, never propose dropping Czech.

11. **Engine time is pinned.** Reason about windows with the engine's own "today" (the thread-local today override used by history replays), never with the wall clock.
12. **Keep the repository clean.** Scratch scripts, screenshots and databases go to a temporary directory — including one-off test files: never drop a scratch test into backend/tests/.
13. **Be honest about coverage.** List every check that could not run and why.

2. Product decisions — do not report as bugs

As of 5 Oct 2026. Check the latest engine documentation for newer decisions before relying on this list.

Area	Decision
Engines	Three engines coexist: v1 Standardní, v2 Citlivý (calibrated sensitive mechanics), v3 Kapacitní (v2 mechanics + capacity-based Zátěž axis + Trénink tab). New runners start on v3 (NEW_RUNNER_ENGINE) ; existing accounts keep their mode. The engine picker on the Data page is owner-only — runners do not switch engines themselves; the switch API stays for the owner and tests.
Dnes layout	The quadrant is the first element on Dnes. Capacity appears lower on Dnes and in full on Zátěž.
Trénink	A guardrail, not a coach: safe ranges for the day, no periodised plans. The Monday weekly plan shows the week bounded by today's capacity ("at the edge of capacity") — still a guardrail: never above the ceilings, paused by any stop or physio referral. An AI coach may come later.
Quality sessions	Allowed only ≥ 48 h after the last hard session, readiness ≥ 65 % (READY_QUALITY; below 45 % only regeneration), no pain ≥ 3 , Zátěž < 25 , ≤ 45 min in Z4+.
Pain rules	Physio override ("no run, book the physio") only when pain $> 5/10$ AND the engine's own physio referral (critical quadrant, alert tier or silent drift). Pain 3–5 modifies the day (shorter, easier, flat/soft, no long run or quality). Pain > 5 without a referral → regeneration or cross-training, no physio push.
Intensity	HR zones from heart-rate reserve (Karvonen); pace together with HR as relative effort against the runner's last 8 weeks.
Calibration	v2/v3 mechanics: ≈ 5 % chance per metric of a shown signal with no real change; the "mechanika přetrvává" flag ≈ 5 %. ACWR is context only. In v3, HRV / resting HR / sleep act through readiness, not as separate load points.
Pace	Per-run mechanics are adjusted to the runner's own pace sensitivity in all engines (v1 changed deliberately).
Profile gate	Since v0.12.0 accounts without birth year, sex and running start (running_since) are asked for them before using the app — older accounts included, on purpose. Seeded demo runners carry these fields.
Roles & accounts	Public registration creates runners only; physio, employer and partner accounts are created by the owner. A physio sees only runners who opted in (physio_interest) and whose case they claimed; a runner can revoke a physio. The owner's "Zobrazit jako" view is read-only and logged.
AI features	External LLM calls: the physio AI brief (claimed runners), and the runner's own AI features (coach texts, per-tab assistant, report AI notes) only with the runner's AI consent (coach_consent); shoe-photo recognition only when the runner starts it. Output is labelled as AI with a rule-based fallback.
Regulatory	Došlap is not a medical device (MDR): monitoring and decision support, no diagnosis or treatment language; pain is routed to a physiotherapist. Every guidance and risk view shows the not-a-medical-device note. Self-management exercise programmes are general strengthening for a marked area, never treatment of a named diagnosis.

3. Orientation — what the app is

- **Architecture:** one container. FastAPI backend (SQLAlchemy; SQLite locally, Postgres when DATABASE_URL is set) serves a React + Vite single-page app from the same origin; CSRF protection is a same-origin check. Czech UI with a British English translation layer: Czech strings are the keys (scripts/i18n/en/*.json → frontend/src/i18n/en.json via scripts/i18n/merge.py).
- **Roles:** runner, physio, employer, partner, plus the owner (admin view, engine tools). The UI offers runner sign-in only.
- **Data in:** Garmin (export import, live sync with stored tokens and a pre-07:00 auto-sync, per-second detail streams, all-day heart rate, ride/swim HR traces), Apple Health (import and push webhook, sleep stages, night HRV), self-

report (check-ins, run ratings with pain points, OSTRC injury reports, shoes, races, profile incl. menstrual cycle).

- **Engine out:** an assessment with three axes (mechanika / zátěž / příznaky), overall score, tier, quadrant (threshold 25 with hysteresis 18), a ranked signal list, triage decision, capacity picture and (v3) daily guidance plus readiness, critical speed (Rychlost), illness and cycle-aware signals. History views replay the engine day by day on a pure in-memory snapshot of the runner's data (RunnerData.as_of).
- **Reports:** morning/evening report cards, the Monday weekly plan, optional AI notes.
- **Tools around it:** Excel backtests (v1 vs v2), a sensitivity sandbox that mirrors the scoring, signal documentation, a physio AI brief, a per-tab AI assistant with evidence cards, self-management exercise programmes.

4. Locate by role — discovery map

Search for the pattern (git grep / ripgrep) instead of trusting the hint. Record anything that moved.

Role	Discovery search	Hint (5 Oct 2026)
App entry, routers, SPA serving	FastAPI(· include_router(· StaticFiles	backend/app/main.py
Additive DB migrations	def add(· ALTER TABLE (near startup)	backend/app/main.py (_migrate)
Data model	class .*(Base)	backend/app/models.py (48 models)
Auth, session, CSRF	verify_csrf · same_origin · SESSION_COOKIE	app/deps.py, app/security.py, routers/auth.py
Access control	ensure_runner_read_access · ensure_runner_self · require_role · require_owner	app/deps.py, routers/admin.py
Access log (GDPR)	AccessLog · _record_access	app/main.py, app/deps.py
Engine core / axes	def assess(· def recompute_assessment(	app/metrics/engine.py
Engine inputs (the one read)	def load_runner_data · def as_of · {{TBD-INPUTS}}	app/metrics/data.py
Engine modes	_emode · _sensitive · engine_pinned · engine_version_for · _MODE_SUFFIX · NEW_RUNNER_ENGINE	app/metrics/engine.py, routers/auth.py
Per-run mechanics drift	_drift_z_core · _pace_slope · _ewma_flag · _mech_flags	app/metrics/engine.py
Segment scoring	segment_mechanics · _seg_scale · segment_significance · t_two_sided_p · _seg_fdr	engine.py, segmentation.py, regression.py
Stream QC	sample_weights · parse_garmin_details · SEG_VERSION	stream_qc.py, segmentation.py
Capacity / v3 load axis	def assess_capacity · CHANNELS · band_points	app/metrics/capacity.py
Critical speed / Rychlost	def critical_speed · mean_max · _CS_CACHE	app/metrics/speed.py
Runner factors (novice, shoes)	shoe_state · TRANSITION_DAYS · novice	app/metrics/runner_factors.py
Illness / cycle / day load	illness · cycle · dayload	app/metrics/illness.py, cycle.py, dayload.py
Training guidance	def build_guidance · TYPE_LABEL · READY_QUALITY	app/metrics/guidance.py
Reports & weekly plan	_rest_of_week · week_plan · report_ai	app/metrics/daily_report.py, week_plan.py, report_ai.py

Terrain load	minetti_cr · load_km · descent_weighting	app/metrics/terrain.py
Triage / referral	def triage_decision · DECISION_HEAD · def claim(	app/metrics/engine.py, routers/triage.py
History replay	def engine_replay · def load_inputs · input_digests · today_pinned(	app/history.py; Excel backtest: app/reports.py (_replay_both, in-memory SQLite, v1/v2 only)
History cache	EngineHistoryCache · HISTORY_VERSION · mark_dirty	app/history.py
Metric caches	lru_cache · _CACHE	app/metrics/*.py (capacity, speed, engine)
Sensitivity sandbox	def simulate(· KNOBS · inputs_from_assessment	metrics/sensitivity.py, enginelab.tsx
Signal documentation	SIG_DOC · METRIC_INFO	metrics/sig_doc.py, frontend/src/metricinfo.ts
Garmin / streams	fetch_details · _fetch_streams · auto_sync_all · _apply_seed · _merge_seed	backend/garmin_live.py, routers/integrations.py
External LLM	def chat(· def vision(· NVIDIA_ · transcribe · coach_consent	app/llm.py (callers: metrics/ai_brief.py, coach_texts.py, report_ai.py, app/assistant/service.py, app/shoes.py)
Self-management programmes	PROGRAMS · programs_for_regions · "match"	app/programs_library.py, routers/self_programs.py, selfprograms.tsx
Frontend routes / nav	createBrowserRouter · runnerNav · useRunnerNav	frontend/src/App.tsx, shell.tsx
Frontend views	export function (Load Mechanics Training Care DataView EngineLab Report)	tabs.tsx, training.tsx, capacity.tsx, care.tsx, datapage.tsx, enginelab.tsx, report.tsx
Tests & fixtures	conftest · seed_runs · DEMO_PASSWORD	backend/tests (conftest.py, synth.py)

5. Full review — step by step

Run the phases in order. Each item is a check with its pass condition. Record PASS / FAIL / NOT RUN for every item in the report. Phases are independent enough to run in parallel (e.g. sub-agents for E+G, F, H, I) on separate throwaway databases and ports; the lead re-verifies every S1/S2 before reporting it.

Phase A — Baseline

1. **A1 What changed:** git status, git log since the last report in Documentation/reviews/ (on claudereviews), diff stat. Focus on changed areas first. If the last report's commit is unreachable (history rewritten), say so and use `git log --since=<report date>`.
2. **A2 Environment:** Python virtualenv (Python ≥ 3.12 ; `.python-version` pins 3.13) and Node dependencies present; note versions.
3. **A3 Backend tests:** run the full suite. Pass: all green. Record count, skips and duration (`{{TBD-TESTS}}`). A duration jump $> 2\times$ without new tests is a finding (or machine load — rerun once).
4. **A4 Frontend:** type-check and production build must succeed with no errors.
5. **A5 Python import sanity:** `python -m compileall` all backend modules (catches syntax errors in scripts tests don't import).

Phase B — Map and structural drift

1. **B1** Run every discovery search from section 4; record the current location of each role.
2. **B2** List routers and count endpoints; list models and columns; list frontend routes and tabs. Compare with the previous report — new endpoints, tables, columns, tabs and engine modes need the targeted checks in phases C–H.
3. **B3** Note any role that disappeared or split; propose the playbook update.

Phase C — Engine correctness (highest priority)

1. **C1 Mode plumbing.** Every engine mode appears consistently in: the switch endpoint's allowed list (unknown modes are rejected), the version suffix map and its reverse lookup, the frontend mode union types, the Data-page options, the v3-only Trénink nav gate, the history cache key. Search for direct comparisons of the engine mode (e.g. `_emode() ==`) across all of backend/app; each must be intentional — “sensitive mechanics” must use the shared helper that includes v2 and v3 (`_sensitive()`).
2. **C2 Time.** Inside the engine and metrics code, no wall-clock calls (`datetime.now`, `date.today`, `time.time`) except the engine's own today helpers and the documented weekly-frozen population priors (`reference._real_today`). Windows use the engine's day helpers; date strings are compared on the date part (`[[:10]]`).
3. **C3 Replay parity.** Start from the one read, `load_runner_data` (section 4): every table it loads is an engine input and must reach both the replay (via `as_of`) and the history-cache digest, so a change to it invalidates the cached history. Pass: for v1, v2 and v3 — including a runner with stored streams and a v3 runner with a shoe — the history's last point equals the live assessment on all axes and the quadrant. Also switch a runner's engine with mechanics in the 18–25 hysteresis band and compare live vs history.
4. **C4 Caches.** History cache keyed by engine version + history version and invalidated on every recompute. Every module-level cache in the metrics code (`lru_cache`, `...CACHE`) must key on everything that changes its result — a data fingerprint (segment count, stream fetch time, profile hash), never ids alone. Probe: compute, mutate the data behind the same ids, recompute in the same process.
5. **C5 Direction and units.** Every scored mechanics metric has a risk direction; scores and v2/v3 flags are one-sided (an improvement never raises a warning); coupled metrics (vertical ratio + oscillation; cadence + stride) count once for convergence. Per-run and segment drift are both in session typical-error units.
6. **C6 No rounded values in scoring.** Values rounded for display must never feed back into points (use exact fields).
7. **C7 Capacity invariants (v3).** Per-run ceiling = capacity $\times (1 + \text{margin}) \times \text{readiness}$; readiness factor within 0.70–1.00; injury history applied once (margins), not also as a multiplier; runs followed by pain $\geq 3/10$ within 72 h excluded

from capacity; ≥ 3 prior runs in 30 days before a per-run capacity exists; ≥ 4 weeks before a weekly one. Margin narrowing factors (novice year, new-shoe transition, under-conditioning) never push the margin below SHRINK_MIN.

8. **C8 Guidance invariants (v3).** Every type's $\text{km} \leq \min(\text{per-run ceiling, weekly budget left})$; running exactly the ceiling adds no load points; overrides exactly as in section 2; quality only per section 2 (readiness $\geq 65\%$); taper and deload applied; "předběžné" until today's sleep/HRV; paces inside the runner's own observed speed range; ≤ 45 min in Z4+ per quality session. The evening report's "Zítra" and "Zbytek týdne" and the weekly plan obey the same overrides.
9. **C9 Sandbox parity.** For every engine mode, seeding the sandbox from a live assessment reproduces the live load and mechanics axes exactly. Every new signal must exist in both places.
10. **C10 Statistics.** p-values from Student's t with the right degrees of freedom; FDR on unrounded p; context model (speed, gradient, time, surface) used where available; out-of-domain segments not scored; minimum sample sizes respected.
11. **C11 Stream pipeline.** Durations from timestamps, never row counts; QC acceptance enforced; units normalised; any change to segmentation bumps the segmentation version so stored streams are re-fetched.
12. **C12 Numerical robustness.** Division by zero, None / empty inputs, zero or negative durations, profiles with < 2 points, HR max $\leq$ resting HR, NaN. Probe with degenerate synthetic runners (one run; no HR; no daily data; identical values).
13. **C13 Performance.** Live assessment ≤ 0.3 s for a 6-month runner with streams; a 183-day history replay ≤ 20 s in every mode (v3 is the slowest). Look for quadratic work inside per-segment or per-session loops and for recomputation a cache should cover.
14. **C14 Docs match code.** Formulas and constants in docstrings, signal documentation, info texts and the engine documentation PDF equal the constants in code (check ~15 constants each review, incl. readiness bands, ACWR bands, pain thresholds, windows).

Phase D — Calibration and behaviour on data

Behavioural targets. Use synthetic runners driven through the real engine functions (recipe in appendix B) and the seeded demo runners replayed day by day. Compare with the previous report; any shift needs an explanation. Use $N \geq 1000$ (or several seeds) for rates near a threshold — at $N = 300$ one standard error is ≈ 1.3 points.

Check	Target (5 Oct 2026)	Fail if
Per-metric shown signal ($z \geq 0.6$), no real change	$v1 \approx 3.5\%$, $v2/v3 \approx 5\%$	$v2 > 8\%$
"mechanika přetrvává" flag, no real change	$\approx 5\%$	$> 8\%$
That flag when a metric improves by 1 SD	0 %	$> 1\%$
Easy runs 25 s/km slower, form unchanged	"mechanika přetrvává" flag $\leq 5\%$ ($v1$, $v2$, $v3$)	$> 10\%$
Detection: +1 SD over the last 14 days ($v2$)	$\approx 67\%$	$< 55\%$
Segment session drift SD, no real change	0.8–1.25	outside
Segment significance false positives, 5-segment baseline	$\approx 5\%$	$> 7.5\%$
Demo accounts today ($v3$)	Tichý drift = silent; Kritické přetížení = critical	different
Stable demo runners, 90-day replay ($v3$)	Zátěž ≥ 25 on $\leq -30\%$ of days	big unexplained change
Physio override in guidance	only pain > 5 + referral	any other trigger

Phase E — Data ingestion and integrations

1. **E1 Imports and syncs are idempotent:** re-importing / re-syncing never duplicates activities or daily metrics. A file re-import never deletes runner-entered data (manual activities, run ratings and pain points, exclusions, hand-edited days) or another provider's history. Run import tests with SQLite foreign keys enforced (or on Postgres).

2. **E2** Garmin rate limits (HTTP 429) stop the batch without marking runs as failed; genuine failures are marked so the backfill advances; stale streams are re-fetched; refresh failures keep the old data (running and HR-only streams); a failed HRV / resting-HR range call is filled in by a later sync.
3. **E3** Stored tokens are encrypted when the secret is configured; a revoked token disables auto-sync with a visible error; a transient Garmin error (network, 5xx) never disables auto-sync; every auto-sync failure is written to the runner's last error.
4. **E4** The daily auto-sync runs in Europe/Prague time before 07:00 (time-zone-aware, independent of the container TZ, across DST), and failures for one runner never stop the others.
5. **E5** Apple push webhook requires its token; unknown payloads are rejected with 400, not partially stored and never 500; the token never appears in logs; sleep and steps are de-duplicated across sources like the file import.
6. **E6** Cross-training reaches the load axis but never the running mechanics.

Phase F — API, security and privacy

1. **F1** Every mutating route (POST / PATCH / PUT / DELETE) carries the CSRF dependency, or is a documented exemption. Known exemptions (5 Oct 2026): the sandbox's simulate and sweep (stateless), the Apple push webhook (token), operator-token routes guarded by `require_feedback_token` (annotations export/resolve, assistant ops/*). Known accepted item: logout has no CSRF check (S3 — a cross-site request can only log the user out). Probe a foreign Origin, Origin "null", a look-alike host, a foreign Referer without Origin, and no Origin/Referer at all. {{TBD-CSRF}}
2. **F2** Every runner-scoped route calls a runner access check (read access, self) or a role requirement, and every nested object id is checked to belong to that runner. Probe IDOR with two runner accounts and a physio who has not claimed the runner: each must get 403/404 for foreign data. Probe the physio claim: claiming a closed case, a runner without consent, another physio's case or after the runner revoked you → refused; public registration with a non-runner role → refused.
3. **F3** Session cookie: HttpOnly, Secure over HTTPS, SameSite; argon2 password hashing; login throttle active and not bypassable by rotating X-Forwarded-For; session tokens stored hashed; "log out all devices" revokes every session.
4. **F4** The JSON export and logs contain no passwords, tokens or session secrets; secret files stay git-ignored and docker-ignored.
5. **F5** External LLM calls happen only as in section 2 (AI features), send the minimum data, have timeouts and a fallback, and are labelled as AI output. For each call site, check the consent gate with a mocked LLM that must not be called without consent.
6. **F6** Uploads: size limits and zip handling (no path traversal, no zip bombs — check the uncompressed size before extracting), malformed files return a clear 400.
7. **F7** GDPR: physio and owner access to a runner is written to the access log and visible to the runner — including routes outside `/api/runners/{rid}/` (AI brief/chat, RTR, conclusions, physio bootstrap, sandbox inputs).

Phase G — Persistence, migrations, deployment

1. **G1** Every new or changed model column has an additive startup migration that works on SQLite and Postgres, with a backfill where old rows need a value.
2. **G2** Seeding is idempotent and never overwrites existing accounts; demo accounts exist on already-seeded databases.
3. **G3** The container builds (frontend then backend); the health endpoint reports database persistence honestly; DEPLOY.md lists every environment variable the code reads (search `os.environ` / `getenv`).

Phase H — Frontend and UX

Drive the real app in a headless browser (appendix A) for at least: one v1 demo runner and one v3 demo runner, at 1360 px and 400 px width, browser locale `cs-CZ`, time zone `Europe/Prague`. Before full-page screenshots, inject CSS that disables animations — the app uses scroll-driven reveal animations, so off-screen content otherwise renders transparent. That artefact is not a bug.

1. **H1** Every tab loads with no console errors and no horizontal overflow (`scrollWidth - clientWidth = 0`) on both widths.

2. **H2** The Trénink tab appears only for v3 runners, next to Dnes; switching engines on the Data page (owner only) updates navigation.
3. **H3** Consistency: the quadrant on Dnes, the Zátěž and Pohyb headlines, the last point of each history chart and the morning/evening/weekly reports agree with the live assessment and guidance.
4. **H4** Numbers: decimal comma, sensible rounding, paces as m:ss (never “5:60”), Czech dates, units always shown.
5. **H5** Czech copy: diacritics, prepositions (v / ve), formal “vy”, no English leftovers, consistent terms (Zátěž, Pohyb, Příznaky, Kapacita, Přípravenost, norma); every new metric has an info text. One pass with English selected: no untranslated Czech.
6. **H6** Empty, loading and error states render (new runner with no data; network error). A failed load never shows a reassuring state (no default “Stabilní”, score or tier).
7. **H7** Tap targets ≥ 20 px on phones; text contrast readable on the dark theme.

Known environment noise in the cloud sandbox: fonts.googleapis.com fails with a certificate error; a native date input follows the browser's UI locale (mm/dd/yyyy in an en-US Chromium) — neither is an app defect.

Phase I — Content, evidence and regulatory

1. **I1** No diagnosis or treatment claims; every guidance and risk view carries the not-a-medical-device note; pain always routes to a physiotherapist, never to a diagnosis. Self-management programmes: matched to the marked area exactly (no substring matches across regions), no diagnosis links or cure wording, with a referral note for red flags. Check the English translations too.
2. **I2** Evidence grades (A/B/C) match the cited evidence; every reference is verifiable and correctly described (design, population, sample size, significance — a feasibility study or a non-significant result must not be described as establishing a fact; a working assumption is labelled as the app's, not attributed to a study).
3. **I3** The engine documentation in Documentation/ reflects current behaviour and version; stale sections are findings.

Phase J — Improvement proposals

- Propose an improvement only with: the observed problem and its evidence, the expected benefit (safety, validity, clarity, speed), cost and risk, and how to validate it (test, simulation or backtest).
- Prefer changes that improve safety and measurement validity over cosmetic ones; avoid churn.
- Separate **quick wins** (< 1 hour, low risk) from **larger work**; flag anything that changes scores for existing runners (needs an engine version bump and a calibration run).

6. Regression watchlist — defects found before

Re-check each on every full review; they are the most likely to come back when code is edited.

Past defect	How to re-check
History replay omitted a table (segment streams) → history ≠ live for v2/v3	Replay-parity check (C3) with streams, all modes
Warning flags two-sided; σ taken from the recent sample → tiny consistent drifts flagged	Improvement probe (flag = 0 %); tiny-consistent probe (no flag)
Garmin detail streams decimated; rows counted as seconds → long runs lost	Decimated synthetic stream (4–5 s/row) still yields ~60 s segments
QC acceptance computed but not enforced	Short/low-coverage stream stores no segments
Regression domain ignored surface	Unseen surface is not scored
Significance test used raw buckets and normal p with tiny baselines	FPR ≈ 5 % at n = 5; a faster run isn't flagged
Pace confound: slower easy runs read as mechanical drift	Pace +25 s/km probe on the flag, all modes (phase D)
Noise scale too small (MAD with floor), λ uncalibrated	Null false-signal rates (phase D)
Segment drift in the wrong units	Null segment DI SD 0.8–1.25
Quadratic median inside the regression fit	Replay timing; profile hot paths
Rounded ratio used for points	Search scoring code for round() before band / points
Sandbox diverged from the engine when a mode was added	Sandbox parity test for every mode
Tests switched a shared demo runner's engine, breaking other tests	Tests register their own runners; no test mutates seeded accounts
Pace display “5:60”	Format 359.6 s/km → 6:00
Czech number formatting / preposition errors in generated text	Grep generated strings for “.” decimals (incl. capacity ratios and the sandbox)
A citation overstated its evidence (small feasibility study)	Reference audit (phase I)
A mode gate compared <code>_emode() == "v2"</code> instead of <code>_sensitive()</code> → v3 lost terrain-weighted spikes (2026-10-05)	C1 search across backend/app; v3 terrain-spike test
New engine input (shoes, RTR plans) loaded live but missing from the history digest → history ≠ live (2026-10-05)	C3 parity test that mutates every input table
Live hysteresis seeded from the previous engine's stored quadrant after a switch (2026-10-05)	Switch engines with mechanics in 18–25; live = history
Metric cache keyed on ids only → critical speed frozen before streams arrived (2026-10-05)	C4 mutate-then-recompute probe for every module cache
File re-import deleted runner-entered data; FK violations only on Postgres (2026-10-05)	E1 re-import test with FK enforcement
Any self-registered physio could claim any runner (2026-10-05)	F2 claim probes; public register with role physio
An AI feature sent health data without the AI consent (2026-10-05)	F5 mocked-LLM consent test per call site
Load failure rendered “Stabilní · Skóre 100 · nízké riziko” (2026-10-05)	H6 route-abort check on Dnes
Evening report planned “kvalitní” days during a physio stop (2026-10-05)	C8/H3 report test with pain 8/10 + referral
Programme region matched by substring (upper abdomen → lower-abdomen programme) (2026-10-05)	I1 region → programme map test
One Garmin outage disabled auto-sync for everyone (2026-10-05)	E3 resume with a ConnectionError keeps auto-sync on

7. Severity and confidence

Severity	Meaning in Došlap
S1 Critical	Could lead a runner to unsafe training or suppress a needed physio referral; security or privacy breach; data loss or corruption; app down.
S2 Major	Wrong scores or guidance in common situations; live and history disagree; calibration off target; a feature broken for a role or engine.
S3 Minor	Edge cases, inconsistent copy or formatting, performance regression without user impact.
S4 Improvement	No defect — a proposal (phase J).

Confidence	Meaning
CONFIRMED	Reproduced (failing test, script output, screenshot).
LIKELY	Strong code-path evidence, not reproduced.
PLAUSIBLE	Needs verification; say what would confirm or refute it.

8. Deciding what is a finding

- Report behaviour that breaks a contract in this playbook, a test, the engine documentation or a product decision — or that a runner or physio would reasonably consider wrong.
- Don't report style preferences, hypothetical attacks without a path, or “could be refactored” without a benefit.
- Merge duplicates: one root cause = one finding, listing all symptoms.
- Before reporting, try to disprove the finding (look for a guard elsewhere, a test that covers it, a documented decision). Keep only what survives.

9. Report template

Save as Documentation/reviews/review-YYYY-MM-DD-quick.md or ...-full.md on branch `claude/reviews` when the task allows writing files (create the folder if missing); otherwise return it in the conversation. Commit message: Review YYYY-MM-DD (quick|full): S1 a, S2 b, S3 c, S4 d.

```

# Došlap review – YYYY-MM-DD (quick screen | full review)
[Full review recommended / Fix before the next release – when applicable]
Commit / working tree: <hash>, <uncommitted files noted, untouched>
Baseline: tests <passed/skipped/failed, duration> · type-check <ok> · build <ok>

## Summary
<3–5 lines: overall state, top risks, what changed since the last review>

## Findings (most severe first)
### F1 <short title>
Severity: S1–S3 · Confidence: CONFIRMED | LIKELY | PLAUSIBLE · Area: <phase>
Location: <file:line>
What happens: <one sentence>
Failure scenario: <concrete inputs → wrong output>
Evidence: <test / script output / screenshot / code path>
Suggested fix: <description or diff sketch> Test to add: <name + assertion>
Health-risk direction: false reassurance | false alarm | none

## Fixed since last review
## Product questions
## Improvement proposals (S4) – problem · benefit · cost/risk · validation
## Check results (PASS / FAIL / NOT RUN per check)
## Calibration table vs previous review (phase D)
## Coverage gaps – check · why it could not run
## Structural drift – role · old location → new location · playbook update proposed

```

10. Quick screen (frequent runs)

1. Phase A baseline (tests, type-check, build) and the diff since the last report (if its commit is unreachable: say so, use `git log --since the report date`, and screen at least the last 20 commits).
2. Discovery searches for any role touched by the diff; note drift.
3. Mode plumbing and wall-clock searches (C1, C2); CSRF and access-check listing for new routes (F1, F2) including nested object ids.
4. New model columns vs startup migrations (G1); new engine inputs vs the history digest (C3).
5. Replay parity and sandbox parity tests (C3, C9); guidance override tests.
6. Quick null simulation ($N \approx 300$ synthetic runners) for the per-metric false-signal rate (phase D row 1).
7. One v3 demo runner in the browser at 400 px: Dnes, Trénink, Zátěž — no console errors, no overflow.
8. Regression watchlist items touched by the diff.

Put “Full review recommended” at the top when the screen finds an S1 or two or more S2, or when the diff is too large to screen (e.g. > 50 files).

Appendix A — Command recipes (adapt paths via section 4)

Baseline

```
git status --short && git log --oneline -20
cd backend && python3.13 -m venv .venv && .venv/bin/pip install -q -r requirements.txt # or the
owner's .venv_posix
.venv/bin/python -m pytest -q # full backend suite
.venv/bin/python -m compileall -q app *.py # import sanity incl. scripts
cd ../frontend && npm ci && npx tsc --noEmit -p . && npm run build # type-check + production build
```

Run the app on a throwaway database (the app creates tables and seeds demo data)

```
cd backend
DOSSLAP_DB_PATH=$TMPDIR/review.db DATABASE_URL= DOSSLAP_WEATHER=off DOSSLAP_EMBED=off
DOSSLAP_PRECOMPUTE=0 \
  NVIDIA_API_KEY= LLM_API_KEY= ASSISTANT_API_KEY= EMBED_API_KEY= VISION_API_KEY= \
  .venv/bin/python -m uvicorn app.main:app --port 8765 &
until curl -sf -o /dev/null http://127.0.0.1:8765/; do sleep 1; done
# owner tools (engine picker, account creation): add DOSSLAP_OWNER_EMAILS=adela@demo.cz
# stop: lsof -ti:8765 -sTCP:LISTEN | xargs kill
```

Log in and switch engines (CSRF = same-origin, so send an Origin header)

```
H="Origin: http://127.0.0.1:8765"
curl -s -c jar -H "$H" -H 'Content-Type: application/json' \
  -d '{"email":"kritickepretizeni@demo.cz","password":"demo"}' http://127.0.0.1:8765/api/auth/session
curl -s -b jar -H "$H" -H 'Content-Type: application/json' -d '{"mode":"v3"}' \
  http://127.0.0.1:8765/api/runners/<runner_id>/engine
```

Seeded demo account	Password	Use it to test
adela@demo.cz	demo1234	runner with pain and mechanics drift (v1 by default)
tichydrift@demo.cz	demo	silent drift (mechanics only)
kritickepretizeni@demo.cz	demo	critical overload, physio override in v3 guidance
havlickova@fyzioholesovice.cz	demo1234	physio role (claims, AI brief, IDOR probes)
hexanet@demo.cz / letna@demo.cz	demo1234	employer / partner roles

Extra physios for claim probes are created by the owner ({TBD-ACCOUNTS}); public registration creates runners only.

Search checks

```
grep -n '@router\\.\\(post\\|patch\\|put\\|delete\\)' backend/app/routers/*.py # each needs verify_csrf or
a documented exemption
grep -rE 'datetime\\.\\(now|utcnow\\)\\(|date\\.today\\(|time\\.time\\(' backend/app/metrics
backend/app/history.py # only the today helpers
grep -rE '_emode\\(\\) *==|engine_mode *==' backend/app # each intentional;
sensitive = _sensitive()
grep -rE 'lru_cache|_[A-Z_]*CACHE' backend/app/metrics # each key = data
fingerprint
grep -rn 'os.environ\\|getenv' backend --include=*.py | grep -v venv # vs DEPLOY.md
```

Headless browser

Install playwright-core in a temporary directory (not the repo) and point it at a local Chrome or the cached Playwright Chromium (cloud sandbox: /opt/pw-browsers/chromium-*/chrome-linux/chrome, launch with --no-sandbox; never run playwright install). Per page: log in through the UI (/auth opens in sign-up mode — click “Už máte účet? Přihlásit

se”) or the API with the Origin header, open /app/<tab> (today, training, post, mechanics, load, messages; plus /data), wait for a known heading, add a style tag `*{animation:none!important;transition:none!important}`, then screenshot and read `scrollWidth – clientWidth` and the console errors.

Appendix B — Calibration recipes

Synthetic runner → real drift core. Create ~90 days of runs ($\approx 4/\text{week}$) as simple objects with `started_at`, `distance_km`, `duration_min`, `surface`, `ascent/descent` and the metric (e.g. `vert_ratio_pct = 8.0 + 0.3 \cdot N(0,1)`). Pin the engine’s today and mode, call the per-run drift function over many runners and count how often $z \geq 0.6$ and how often the flag helper returns the flag. Variants: heavy-tailed noise; a shift of +1 SD in the last 14 or 5 days (detection); an improvement of –1 SD (the flag must stay 0); easy pace +25 s/km with metrics following speed (pace confound — `measure the flag over all mechanics metrics, with a matched no-slowdown baseline; helper tests/test_drift_validity.py::_history(slow=25)`).

```
with E.today_pinned(TODAY), E.engine_pinned('v2'):
    r = E._drift_z_core(runs, 'vert_ratio_pct', 28)    # name may change: see section 4
    shown += r['z'] >= 0.6
    flag, watch = E._mech_flags(r_vr, r_gct, r_cad, r_stride, r_vosc)
```

Demo replay: set each seeded runner to v1 then v3, replay the last 90 days with the history replay function, and tabulate the share of days with $Z_{\text{átěž}} \geq 25$, the mean, the maximum and the top load signals. Compare with the previous report and with the calibration table in the engine documentation.

Segment checks: build baseline sessions with a between-session and a within-session noise component and score new null sessions with the segment session function; the standardised drift SD must be 0.8–1.25 for several variance splits.

Parity probes (C3/C4): register a runner, mutate one engine input (add a shoe, back-fill streams, edit a check-in, switch engine), call `recompute_assessment` and `refresh_quadrant_history`, and compare the last history row with the live assessment; repeat in a fresh process to catch stale module caches.